

Lecture 16: Spark Joins

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

ED1, 406A, amitkdhar@iitbhlai

March 18, 2026

Introduction to Joins in Big Data

Introduction to Joins

- A **join** operation brings together two sets of data (left and right) by comparing the value of one or more keys.
- In a traditional RDBMS, joins are relatively straightforward because data resides on a single machine.
- In Apache Spark, joins are fundamentally more complex due to the **distributed nature** of the data.
- A single logical DataFrame/Dataset is partitioned across multiple nodes in the cluster. To join matching keys, Spark must often move data across the network.

The Challenge of Distributed Joins

- **Data Locality:** Rows with the same join keys might be located on different executors.
- **Shuffle:** To perform the join, Spark must physically move data across the network so that rows with matching keys end up on the same executor.
- Shuffles involve disk I/O, data serialization, and network I/O, making them the most expensive operations in Spark.
- Our goal is to choose join types and strategies that minimize shuffling and handle skewed data.

Datasets for our Examples

- We will use two simple datasets for our examples: `employees` and `departments`.
- **Employees:** `emp_id`, `name`, `dept_id`
- **Departments:** `dept_id`, `dept_name`
- We will demonstrate how different joins behave when some employees have no department, and some departments have no employees.

Creating Sample Data (Scala)

```
1 import spark.implicits._
2
3 val employees = Seq(
4   (1, "Alice", Some(10)),
5   (2, "Bob", Some(20)),
6   (3, "Charlie", None),
7   (4, "David", Some(10))
8 ).toDF("emp_id", "name", "dept_id")
9
10 val departments = Seq(
11   (10, "Engineering"),
12   (20, "Sales"),
13   (30, "Marketing")
14 ).toDF("dept_id", "dept_name")
```

Scala DataFrames

Creating Sample Data (Python)

```
1 from pyspark.sql import Row
2
3 employees = spark.createDataFrame([
4     Row(emp_id=1, name="Alice", dept_id=10),
5     Row(emp_id=2, name="Bob", dept_id=20),
6     Row(emp_id=3, name="Charlie", dept_id=None),
7     Row(emp_id=4, name="David", dept_id=10)
8 ])
9
10 departments = spark.createDataFrame([
11     Row(dept_id=10, dept_name="Engineering"),
12     Row(dept_id=20, dept_name="Sales"),
13     Row(dept_id=30, dept_name="Marketing")
14 ])
```

PySpark DataFrames

Join Types

Overview of Join Types

Spark supports several join types, defined by how they handle unmatched rows:

- **Inner Join:** Keep rows with matching keys in both datasets.
- **Outer Joins:** Keep rows that match, plus unmatched rows from one or both datasets.
 - Full Outer, Left Outer, Right Outer.
- **Semi Joins:** Keep rows from the left that have a match in the right (do not include right columns).
- **Anti Joins:** Keep rows from the left that do *not* have a match in the right.
- **Cross Join:** Cartesian product of both datasets.

Inner Joins

- **Behavior:** Evaluates the join expression and keeps only rows where the keys exist in both the left and right datasets.
- This is the **default** join type in Spark.
- Rows with 'null' in the join key will be dropped unless handled explicitly (null $\neq$ null in standard SQL semantics).

Inner Join Example

```
1 // Scala
2 employees.join(departments, "dept_id").show()
3 // Output:
4 // +-----+-----+-----+-----+
5 // |dept_id|emp_id| name| dept_name|
6 // +-----+-----+-----+-----+
7 // |      10|      1|Alice|Engineering|
8 // |      10|      4|David|Engineering|
9 // |      20|      2|  Bob|      Sales|
10 // +-----+-----+-----+-----+
```

Scala Inner Join

```
1 # Python
2 employees.join(departments, "dept_id", "inner").show()
```

Python Inner Join

Outer Joins

- **Full Outer Join:** Retains all rows from both left and right datasets. Unmatched rows are padded with `null` values.
- **Left Outer Join:** Retains all rows from the *left* dataset. Unmatched rows from the right are padded with `null`.
- **Right Outer Join:** Retains all rows from the *right* dataset. Unmatched rows from the left are padded with `null`.

Full Outer Join Example

```
1 employees.join(departments, Seq("dept_id"), "outer").show()
2 // Output:
3 // +-----+-----+-----+-----+
4 // |dept_id|emp_id|  name| dept_name|
5 // +-----+-----+-----+-----+
6 // |    null|    3|Charlie|    null|
7 // |    10|    1|  Alice|Engineering|
8 // |    10|    4|  David|Engineering|
9 // |    20|    2|   Bob|    Sales|
10 // |    30|   null|   null|Marketing|
11 // +-----+-----+-----+-----+
```

Scala Full Outer Join

```
1 employees.join(departments, "dept_id", "outer").show()
```

Python Full Outer Join

Left Outer Join

- Common use case: You have a main fact table (e.g., transactions) and want to enrich it with dimension data (e.g., customer details).
- You want to keep all transactions, even if customer details are missing.

Left Outer Join Example

```
1 employees.join(departments, Seq("dept_id"), "left").show()
2 // Output:
3 // +-----+-----+-----+-----+
4 // |dept_id|emp_id|  name| dept_name|
5 // +-----+-----+-----+-----+
6 // |   null|    3|Charlie|    null|
7 // |    10|    1|  Alice|Engineering|
8 // |    10|    4|  David|Engineering|
9 // |    20|    2|   Bob|    Sales|
10 // +-----+-----+-----+-----+
```

Scala Left Outer Join

```
1 employees.join(departments, "dept_id", "left_outer").show()
```

Python Left Outer Join

Right Outer Join

- Exactly the mirror of Left Outer Join.
- Typically, it is cleaner to swap the DataFrames and use a Left Outer Join to make the code easier to read (main table on the left).

Right Outer Join Example

```
1 employees.join(departments, Seq("dept_id"), "right").show()
2 // Output:
3 // +-----+-----+-----+-----+
4 // |dept_id|emp_id| name| dept_name|
5 // +-----+-----+-----+-----+
6 // |      10|      1|Alice|Engineering|
7 // |      10|      4|David|Engineering|
8 // |      20|      2|  Bob|      Sales|
9 // |      30|    null| null| Marketing|
10 // +-----+-----+-----+-----+
```

Scala Right Outer Join

```
1 employees.join(departments, "dept_id", "right_outer").show()
```

Python Right Outer Join

Left Semi Join

- **Behavior:** Returns rows from the left dataset where a match exists in the right dataset.
- **Crucial difference from Inner Join:** The columns from the right dataset are **not** included in the result.
- Also, if multiple matches exist in the right dataset, the left row is only returned *once* (no duplication).
- Useful for filtering rows based on existence in another table (like IN or EXISTS in SQL).

Left Semi Join Example

```
1 employees.join(departments, Seq("dept_id"), "left_semi").show()
2 // Output (Note: dept_name is missing!):
3 // +-----+-----+-----+
4 // |dept_id|emp_id| name|
5 // +-----+-----+-----+
6 // |      10|      1|Alice|
7 // |      10|      4|David|
8 // |      20|      2|  Bob|
9 // +-----+-----+-----+
```

Scala Left Semi Join

```
1 employees.join(departments, "dept_id", "leftsemi").show()
```

Python Left Semi Join

Left Anti Join

- **Behavior:** Returns rows from the left dataset that **do not** have a match in the right dataset.
- Similar to Left Semi Join, it only returns columns from the left dataset.
- Useful for finding records that are missing from a reference table (like `NOT IN` or `NOT EXISTS` in SQL).

Left Anti Join Example

```
1 employees.join(departments, Seq("dept_id"), "left_anti").show()
2 // Output:
3 // +-----+-----+-----+
4 // |dept_id|emp_id|  name|
5 // +-----+-----+-----+
6 // |   null|     3|Charlie|
7 // +-----+-----+-----+
```

Scala Left Anti Join

```
1 employees.join(departments, "dept_id", "leftanti").show()
```

Python Left Anti Join

Cross (Cartesian) Join

- **Behavior:** Returns the Cartesian product of the two datasets.
- Every row in the left dataset is combined with every row in the right dataset.
- If Left has N rows and Right has M rows, the result has $N \times M$ rows.
- **Warning:** Very expensive! Usually a mistake unless datasets are tiny. Spark requires explicit syntax to perform cross joins to prevent accidental massive shuffles.

Cross Join Example

```
1 employees.crossJoin(departments).show(2)
2 // Output:
3 // +-----+-----+-----+-----+
4 // |emp_id| name|dept_id|dept_id| dept_name|
5 // +-----+-----+-----+-----+
6 // |      1|Alice|      10|      10|Engineering|
7 // |      1|Alice|      10|      20|      Sales|
8 // +-----+-----+-----+-----+
```

Scala Cross Join

```
1 employees.crossJoin(departments).show()
2 # OR
3 spark.conf.set("spark.sql.crossJoin.enabled", "true")
4 employees.join(departments).show() # Without condition
```

Python Cross Join

Join Strategies

Spark SQL Catalyst Optimizer chooses a physical **Join Strategy** to execute the join. The main strategies are:

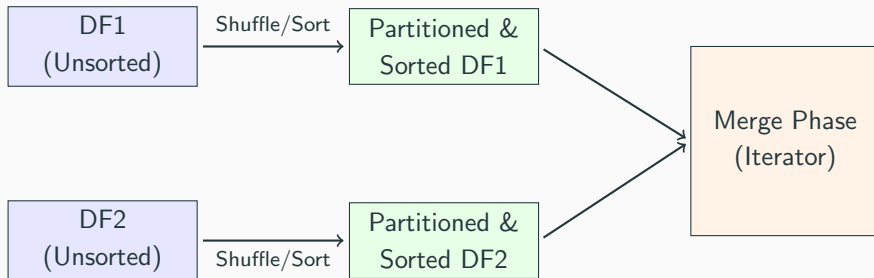
1. **Shuffle Sort Merge Join (SMJ)**: The default for large datasets.
2. **Broadcast Hash Join (BHJ)**: The fastest, used when one dataset is small.
3. **Shuffle Hash Join (SHJ)**: Alternative to SMJ, fast if data is already partitioned correctly.
4. **Cartesian Join**: Used for cross joins.

Strategy 1: Shuffle Sort Merge Join (SMJ)

- This is Spark's default strategy for joining two large DataFrames.
- Consists of two phases:
 1. **Shuffle:** Both datasets are partitioned by the join key. This moves data across the network so that records with the same join keys reside on the same node.
 2. **Sort:** Within each partition, the data is sorted by the join key.
 3. **Merge:** The sorted partitions are iterated over, merging matching keys.

- Because the data is sorted, the merge phase is very efficient (requires only one pass through the data, $O(N)$ time).
- However, the Shuffle and Sort phases are expensive.
- SMJ requires that the join keys be sortable.
- Highly resilient: handles datasets that are too large to fit in memory by spilling to disk.

SMJ Illustration



When is SMJ Used?

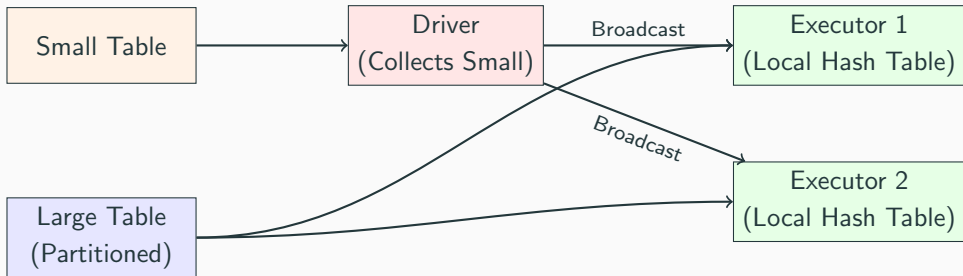
- Used automatically by Spark when joining large tables if `'spark.sql.join.preferSortMergeJoin'` is true (default in Spark 2.x+).
- Fallback mechanism when other joins are not applicable.
- If the tables are already partitioned and sorted (e.g., using Bucketing), the Shuffle and Sort phases can be completely skipped, making SMJ extremely fast!

Strategy 2: Broadcast Hash Join (BHJ)

- The fastest join strategy in Spark.
- **Prerequisite:** One of the datasets must be small enough to fit entirely in the memory of every executor.
- **Mechanism:**
 1. The driver collects the small dataset.
 2. The driver broadcasts (sends a copy of) the small dataset to every executor.
 3. The executors build a local Hash Table from the broadcasted data.
 4. The executors iterate over their partitions of the large dataset and probe the local Hash Table to perform the join.

- **Advantage:** Completely avoids the Shuffle phase! No data from the large dataset is moved across the network.
- **Disadvantage:** The small dataset must fit in driver memory and executor memory. If it's too large, you will get an `OutOfMemoryError`.
- Spark automatically uses BHJ if one dataset's physical size is below `spark.sql.autoBroadcastJoinThreshold` (default is 10 MB).

BHJ Illustration



Enforcing BHJ with Hints

You can explicitly tell Spark to use a Broadcast join using hints, even if the table size is above the threshold (use with caution).

```
1 import org.apache.spark.sql.functions.broadcast
2
3 // Wrap the small dataframe with broadcast()
4 val joinedDF = employees.join(broadcast(departments), "dept_id")
```

Scala Broadcast Hint

```
1 from pyspark.sql.functions import broadcast
2
3 joinedDF = employees.join(broadcast(departments), "dept_id")
```

Python Broadcast Hint

When to Use BHJ

- When joining a massive fact table (e.g., billions of rows) with a small dimension table (e.g., thousands of rows).
- Example: `Transactions JOIN User_Country_Codes`.
- Only supports equi-joins.
- **Warning:** Ensure the broadcasted data actually fits in memory. Spark uses statistics to estimate size, which can sometimes be inaccurate.

Strategy 3: Shuffle Hash Join (SHJ)

- Similar to SMJ, it involves a Shuffle phase.
- Unlike SMJ, it *does not* sort the data.
- **Mechanism:**
 1. Both datasets are shuffled based on the join key.
 2. For each partition, a Hash Table is built from the smaller of the two datasets on that partition.
 3. The larger dataset partition is probed against the Hash Table.

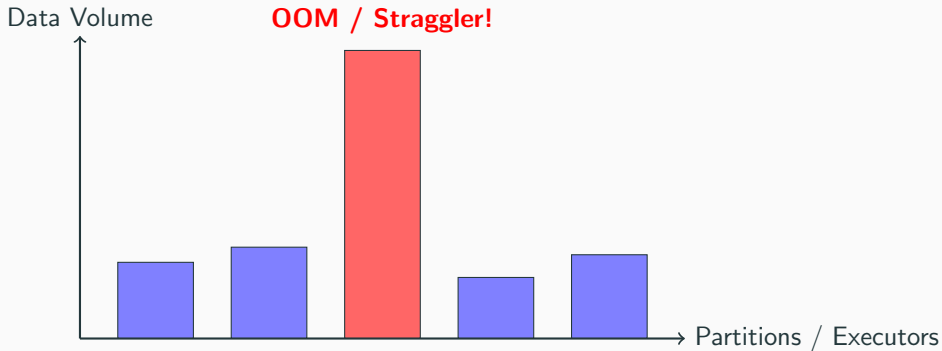
- **Advantage:** Avoids the expensive Sort phase of SMJ.
- **Disadvantage:** Requires the smaller partition to fit entirely in executor memory to build the Hash Table. If data is skewed, one partition might be huge, causing an OOM error.
- Spark generally prefers SMJ over SHJ because SMJ is more robust against OOM errors and handles data skew slightly better (sorting spills to disk).

Handling Data Skew

Handling Data Skew in Joins

- **Data Skew** is the most common cause of Spark job failures and performance bottlenecks.
- Skew occurs when data is not evenly distributed across partitions.
- In a join, if one join key appears millions of times, while others appear only a few times, all rows for the frequent key will be sent to a *single task* during the shuffle.

What is Data Skew?



- Task 3 takes 10x longer to finish than others, or crashes with OutOfMemory. The whole job waits for Task 3.

The Impact of Skew on Joins

- **Straggler Tasks:** 199 tasks finish in 10 seconds, 1 task takes 2 hours.
- **Disk Spill:** The single overloaded executor runs out of memory and spills data to disk, causing massive I/O slowdowns.
- **OOM Errors:** If the executor cannot even process the spilled data, it crashes.
- Skew often happens on "Null" values or default values (e.g., -1, "UNKNOWN").

Detecting Skewed Data

- **Spark UI:** Look at the "Stages" tab. If the *Max* task duration is significantly higher than the *75th percentile* or *Median*, you have skew.
- **Data Analysis:** Group by the join key and count.

```
df.groupBy("join_key").count().orderBy(desc("count")).show()
```

Mitigating Skew: Salting Technique

- **Salting** is a technique to break up a large partition into smaller, evenly distributed partitions.
- We append a random number (the "salt") to the join key in the skewed dataset.
- We replicate the rows in the other dataset for every possible salt value.
- This forces the skewed key to be hashed to multiple different executors during the shuffle.

Salting Implementation (Concept)

Let's say Join Key "A" is highly skewed. We choose a salt range 0...4.

1. Skewed Table (Large):

- Key "A" → becomes "A_0", "A_1", "A_2", "A_3", or "A_4" randomly.
- Skew is spread across 5 partitions.

2. Reference Table (Small):

- Key "A" → must be replicated 5 times: "A_0", "A_1", "A_2", "A_3", "A_4".
- Necessary so that a match can be found regardless of which salt the large table assigned.

Salting Code Example

```
1 import pyspark.sql.functions as F
2
3 # 1. Add random salt (0 to 4) to skewed large table
4 large_skewed = large_table.withColumn(
5     "salted_key",
6     F.concat(F.col("join_key"), F.lit("_"), F.floor(F.rand() * 5))
7 )
8
9 # 2. Replicate small table rows for each salt
10 # Create a dummy array [0, 1, 2, 3, 4] and explode it
11 small_exploded = small_table.withColumn(
12     "salt_array", F.array([F.lit(i) for i in range(5)])
13 ).withColumn("salt", F.explode("salt_array"))
14
15 small_salTED = small_exploded.withColumn(
16     "salted_key", F.concat(F.col("join_key"), F.lit("_"), F.col("salt"))
17 )
18
19 # 3. Join on the new salted_key
20 result = large_skewed.join(small_salTED, "salted_key")
```

PySpark Salting Example

Bucketing

Bucketing to Avoid Shuffles

- What if we need to join two massive tables frequently?
- Both tables are too large for a Broadcast Join.
- A Sort Merge Join will require a massive shuffle *every time* we run the join.
- **Solution: Bucketing.** We can pre-shuffle and pre-sort the data when writing it to disk.

How Bucketing Works

- Bucketing organizes data in the storage system (e.g., Parquet) into a fixed number of "buckets" based on a hash of a column (the join key).
- If both tables are bucketed by the same join key into the same number of buckets, Spark knows that matching keys are already in corresponding files.
- When reading the tables for a join, Spark completely skips the Shuffle phase! This is called a **Shuffle-free Join**.

Bucketing Code Example

```
1 // 1. Write DataFrames as bucketed tables
2 largeDF1.write
3   .bucketBy(50, "user_id")
4   .sortBy("user_id") // Optional but avoids Sort phase too!
5   .saveAsTable("bucketed_table_1")
6
7 largeDF2.write
8   .bucketBy(50, "user_id")
9   .sortBy("user_id")
10  .saveAsTable("bucketed_table_2")
11
12 // 2. Read tables and join (No Shuffle!)
13 val t1 = spark.table("bucketed_table_1")
14 val t2 = spark.table("bucketed_table_2")
15
16 t1.join(t2, "user_id").show() // Extremely fast!
```

Scala Bucketing (Write and Read)

Summary

- Joins are expensive in distributed systems due to network shuffles.
- Choose the right join type (Inner, Outer, Semi, Anti) based on your data needs.
- **Broadcast Hash Join** is the fastest strategy if one dataset is small.
- **Sort Merge Join** is the robust default for large datasets.
- **Data Skew** is the enemy of joins. Use **Salting** to mitigate it.
- **Bucketing** can eliminate shuffles entirely for frequent joins of massive datasets.